Nidharsan V – 22CSR130 III CSE C

Day 3 – Minikube installation and mysql

Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. It helps in efficiently managing multiple containers across a cluster of machines, ensuring high availability, load balancing, and self-healing capabilities. Kubernetes is widely used for cloud-native applications and microservices architectures.

Minikube

Minikube is a lightweight Kubernetes implementation that runs a single-node Kubernetes cluster on a local machine. It is primarily used for development and testing purposes, allowing developers to experiment with Kubernetes features without needing a full-scale cluster. Minikube supports various container runtimes and can be installed on Windows, macOS, and Linux

curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64

sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64

minikube start

minikube start

minikube status

YML file

ersion: '3'

```yaml
services:
  web:
    image: nginx:latest
    ports:
      - 80:80

  db:
    image: mysql:latest
    environment:
      - MYSQL_ROOT_PASSWORD=secret
```

```
docker exec -it david-db-1 /bin/bash
mysql -u root -p
```

Docker compose:

Docker Compose

Docker Compose is a tool that allows you to define and manage multi-container Docker applications using a YAML configuration file (docker-compose.yml). It simplifies the process of running multiple interdependent services (such as a web server, database, and caching system) with a single command.

Key Features:

- **Multi-Container Management** – Define multiple services in one file.
- **Service Dependencies** – Automatically starts services in the correct order.
- **Networking** – Easily creates a shared network for containers.
- **Scalability** – Scale services up or down with a single command.

Example docker-compose.yml:

yaml

Copy code

```yaml
version: '3'
services:
  web:
    image: nginx
    ports:
      - "8080:80"
  db:
    image: mysql
    environment:
      MYSQL_ROOT_PASSWORD: example
```

Usage:

sh

Copy code

```sh
# Start all services
docker compose up -d

# Stop and remove containers
docker compose down
```

Docker compose commands:

```sh
# Start and run containers in the background
docker compose up -d
```

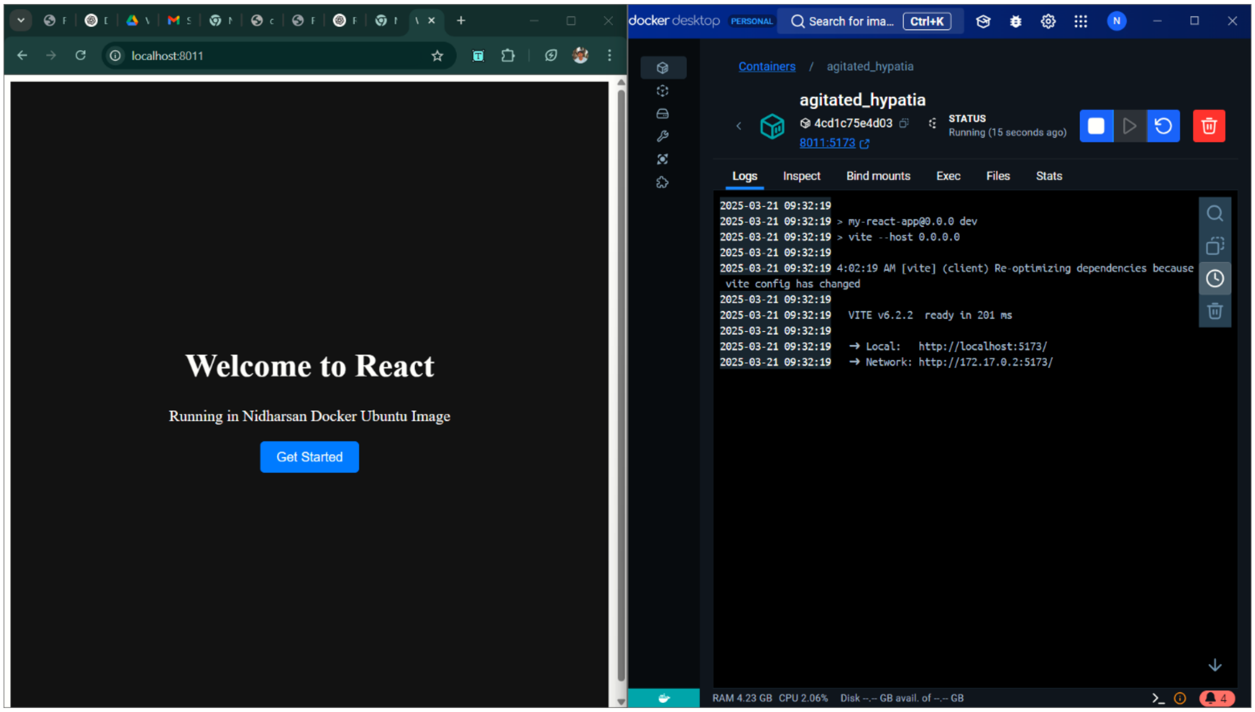
```
# Start containers in the foreground (logs will be shown)
docker compose up

# Stop containers
docker compose down

# Restart containers
docker compose restart

# View running containers
docker compose ps

# View logs of services
docker compose logs

# View logs of a specific service
docker compose logs <service_name>

# Build or rebuild services
docker compose build

# Stop containers without removing them
docker compose stop

# Start stopped containers
docker compose start
```

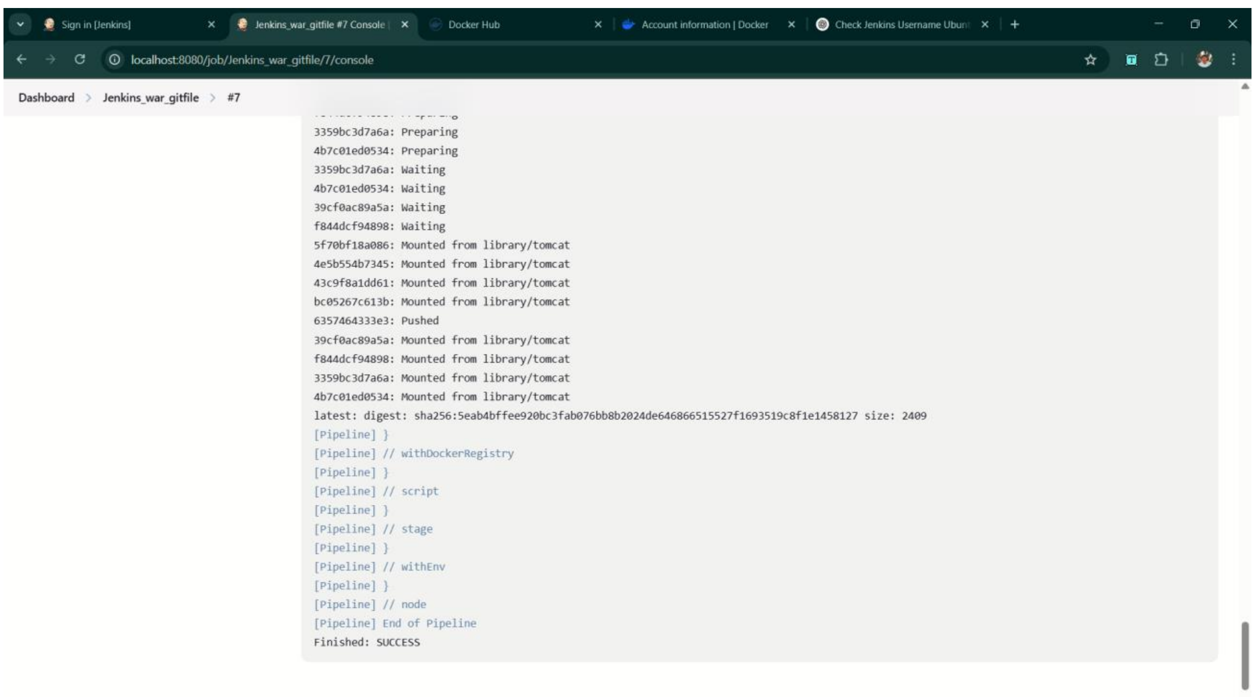
```
# Execute a command in a running container
docker compose exec <service_name> <command>

# Remove stopped containers, networks, and volumes
docker compose down --volumes

# Show configuration details
docker compose config

# Scale a service (e.g., run 3 instances of a service)
docker compose up --scale <service_name>=3 -d
```

Pipeline code

```
pipeline {
    agent any
    tools {maven "maven"}
    stages {
        stage('SCM') {
            steps {
                git branch: 'master', url: 'https://github.com/Saran-Avinash/DevOps.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package'
```

```
            }
        }
        stage('build to images') {
            steps {
                script {
                    sh 'docker build -t nidharsan8008/war-jenkins .'
                }
            }
        }
        stage('push to hub') {
            steps {
                script {
                    withDockerRegistry(credentialsId: 'docker_cred', toolName: 'docker', url:
'https://index.docker.io/v1/') {
                              sh 'docker push nidharsan8008/war-jenkins '
                    }
                }
            }
        }


    }
}
```


localhost:8011
Welcome to React
Running in Nidharsan Docker Ubuntu Image
Get Started
docker desktop PERSONAL
Search for ima... Ctrl+K
Containers / agitated_hypatia
agitated_hypatia
4cd1c75e4d03
8011:5173
STATUS
Running (15 seconds ago)
Logs Inspect Bind mounts Exec Files Stats
2025-03-21 09:32:19
2025-03-21 09:32:19 > my-react-app@0.0.0 dev
2025-03-21 09:32:19 > vite --host 0.0.0.0
2025-03-21 09:32:19
2025-03-21 09:32:19 4:02:19 AM [vite] (client) Re-optimizing dependencies because vite config has changed
2025-03-21 09:32:19
2025-03-21 09:32:19 VITE v6.2.2 ready in 201 ms
2025-03-21 09:32:19
2025-03-21 09:32:19 ➜ Local: http://localhost:5173/
2025-03-21 09:32:19 ➜ Network: http://172.17.0.2:5173/
RAM 4.23 GB CPU 2.06% Disk --.-- GB avail. of --.-- GB


Sign in [Jenkins]
Jenkins_war_gitfile #7 Console
Docker Hub
Account information | Docker
Check Jenkins Username Ubunt
localhost:8080/job/Jenkins_war_gitfile/7/console
Dashboard > Jenkins_war_gitfile > #7
3359bc3d7a6a: Preparing
4b7c01ed0534: Preparing
3359bc3d7a6a: Waiting
4b7c01ed0534: Waiting
39cf0ac89a5a: Waiting
f844dcf94898: Waiting
5f70bf18a086: Mounted from library/tomcat
4e5b554b7345: Mounted from library/tomcat
43c9f8a1dd61: Mounted from library/tomcat
bc05267c613b: Mounted from library/tomcat
6357464333e3: Pushed
39cf0ac89a5a: Mounted from library/tomcat
f844dcf94898: Mounted from library/tomcat
3359bc3d7a6a: Mounted from library/tomcat
4b7c01ed0534: Mounted from library/tomcat
latest: digest: sha256:5eab4bffee920bc3fab076bb8b2024de646866515527f1693519c8f1e1458127 size: 2409
[Pipeline] }
[Pipeline] // withDockerRegistry
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS